Semiconductor Design Capstone Project | 2026-1

Session5 : Combinational Logic Blocks(2)

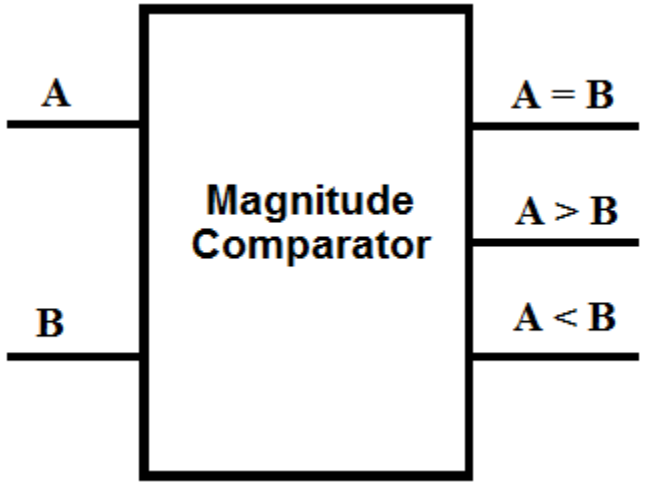
Agenda

- Basic Combinational Blocks(2)
  - Comparator
  - 3 State Bus
- LABS

3. Comparator

Comparator

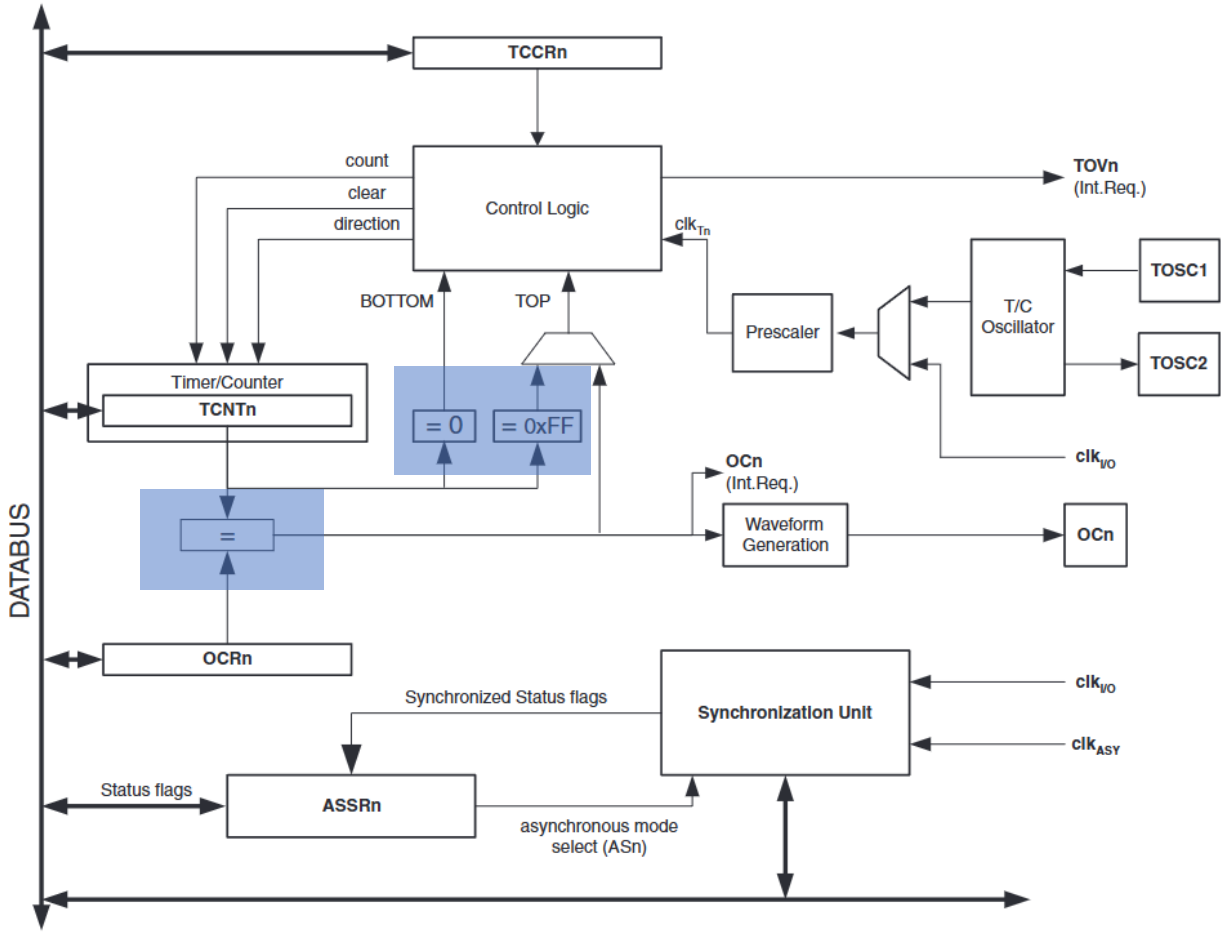
- 비교기 (comparator)
  - 두 개의 n비트 이진수 입력 (A와 B) 크기를 비교
  - 세 개의 출력: A > B, A = B, A < B
  - 동작 원리
    - 입력된 두 이진수를 **비트 단위로 비교**
    - 최상위 비트(MSB)부터 시작하여 하위 비트로 순차적으로 비교
    - 첫 번째로 다른 비트가 발견되면, 그 비트의 값에 따라 크기를 결정



Comparator

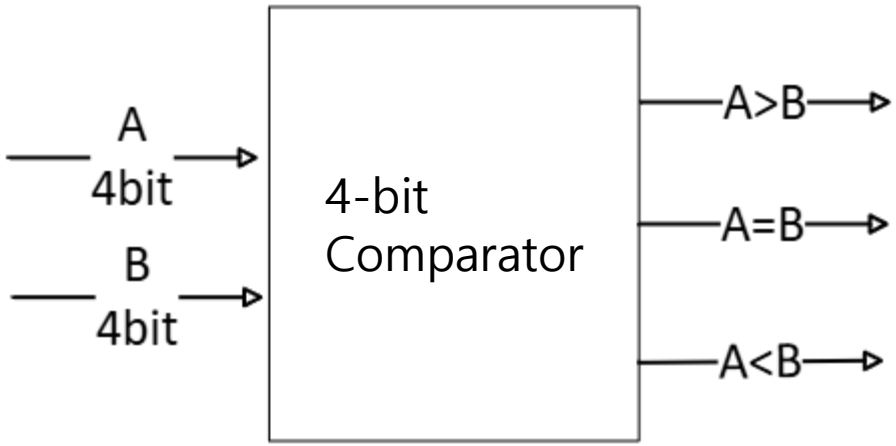
- 데이터 정렬 및 검색:
  - 메모리 관리 유닛(MMU)에서 주소 비교
  - 캐시 메모리의 태그 비교
- ADC 인터페이스:
  - ADC 출력값과 기준값 비교
- 타이머/카운터 모듈:
  - 카운트 값과 설정값 비교
- 전력 관리:
  - 전압 레벨 모니터링 및 비교

Figure 53. 8-bit Timer/Counter Block Diagram



LAB : Specification

- 4-bit Comparator



LAB : DUT

```
`timescale 1ns / 1ps
//comparator.v
module comparator #(parameter WIDTH = 4)(
    input [WIDTH-1:0] a, b,
    output equal, greater, less
);
    assign equal = (a == b);
    assign greater = (a > b);
    assign less = (a < b);
endmodule
```

LAB : Testbench

```
`timescale 1ns / 1ps
//tb_comparator.v

module tb_comparator_4bit();

  // 입력 및 출력 신호 정의
  reg [3:0] a, b;
  wire greater, equal, less;

  // 테스트할 comparator 인스턴스 생성
  comparator uut (
    .a(a),
    .b(b),
    .greater(greater),
    .equal(equal),
    .less(less)
  );

  // 테스트 시나리오
  initial begin
    $monitor("%0t\t %b\t %b\t %b\t %b\t %b", $time, a, b, greater, equal, less);

    // 테스트 케이스
    a = 4'b0000; b = 4'b0000; #10; // 같음
    a = 4'b0101; b = 4'b0011; #10; // A > B
    a = 4'b0010; b = 4'b1000; #10; // A < B
    a = 4'b1111; b = 4'b1111; #10; // 같음 (최대값)
    a = 4'b1010; b = 4'b0101; #10; // A > B
    a = 4'b0111; b = 4'b1001; #10; // A < B
    a = 4'b1100; b = 4'b1100; #10; // 같음
    a = 4'b0000; b = 4'b1111; #10; // A < B (최소값 vs 최대값)

    // 시뮬레이션 종료
    #10 $finish;
  end

endmodule
```

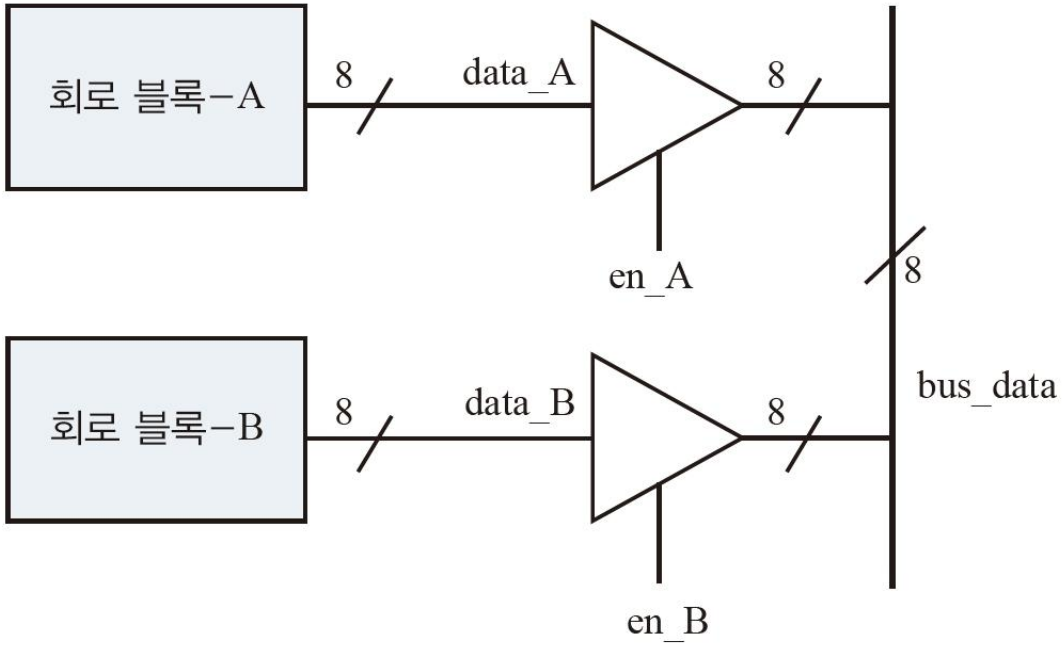
LAB : Waveform

- Wave Form

4. 3-State Bus

Tri State Bus

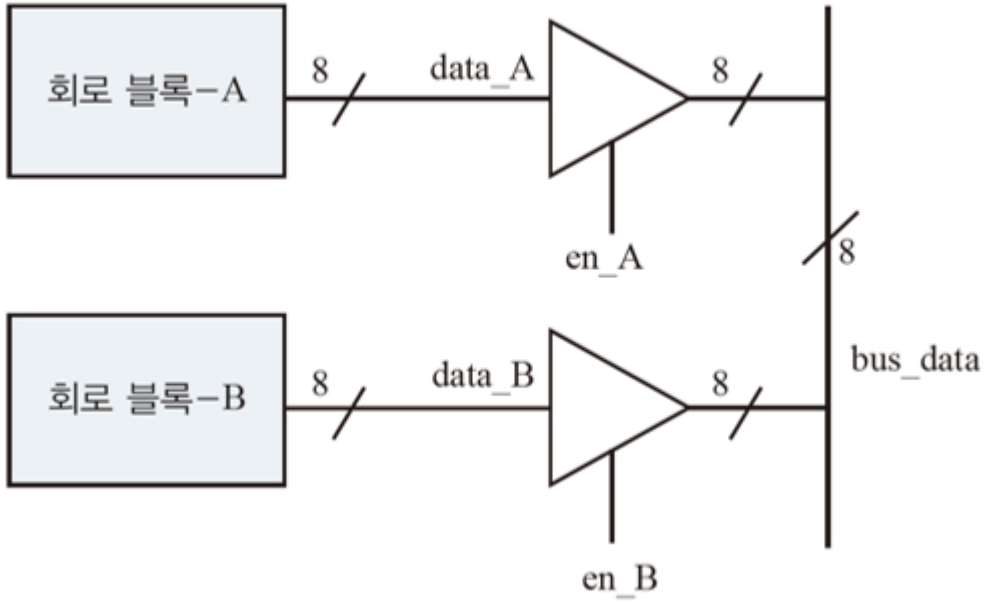
- 3상태 버스 드라이버
  - 다수의 신호원들이 공동으로 사용하는 버스 (bus)에 데이터를 보내거나 또는 신호원과 버스를 격리시켜 high-impedance 상태로 만드는 회로



[그림 10–14] 데이터 버스 드라이버

LAB1 : Specification

- Data Bus Driver



LAB1 : DUT

```
`timescale 1ns / 1ps

module data_bus_driver (
    input [7:0] data_A,     // 회로 블록 A의 데이터
    input [7:0] data_B,     // 회로 블록 B의 데이터
    input en_A,             // 회로 블록 A의 활성화 신호
    input en_B,             // 회로 블록 B의 활성화 신호
    output [7:0] bus_data   // 공통 데이터 버스
);

    // 3상태 버퍼 구현
    assign bus_data = (en_A) ? data_A :
                      (en_B) ? data_B :
                      8'bz;  // 고임피던스 상태

endmodule
```

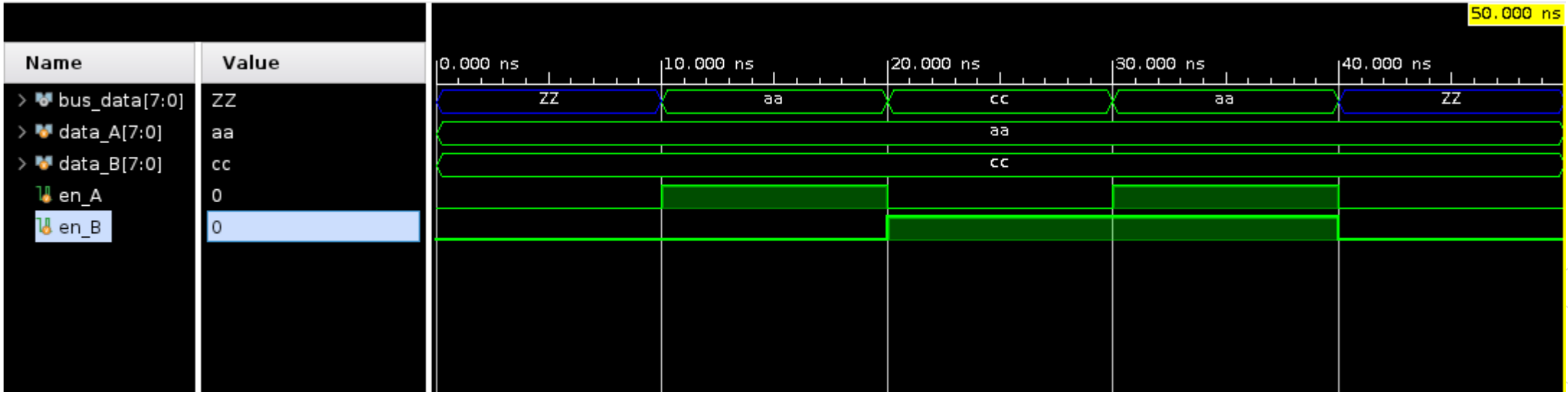
LAB1 : Testbench

```
`timescale 1ns / 1ps

module tb_data_bus_driver();
    reg [7:0] data_A;       // 테스트용 회로 블록 A 데이터 입력
    reg [7:0] data_B;       // 테스트용 회로 블록 B 데이터 입력
    reg en_A;               // 테스트용 회로 블록 A 활성화 신호
    reg en_B;               // 테스트용 회로 블록 B 활성화 신호
    wire [7:0] bus_data;    // 공통 데이터 버스 출력

    // DUT(Design Under Test) 인스턴스 생성
    data_bus_driver uut (
        .data_A(data_A),
        .data_B(data_B),
        .en_A(en_A),
        .en_B(en_B),
        .bus_data(bus_data)
    );
    initial begin
        $monitor("Time=%0t | en_A=%b, data_A=%b | en_B=%b, data_B=%b | bus_data=%b",
                 $time, en_A, data_A, en_B, data_B, bus_data);
        // 초기값 설정
        data_A = 8'b10101010; data_B = 8'b11001100;
        en_A = 0; en_B = 0; #10; // 둘 다 비활성화 -> bus_data는 고임피던스 상태
        en_A = 1; en_B = 0; #10; // A 활성화 -> bus_data는 data_A 출력
        en_A = 0; en_B = 1; #10; // B 활성화 -> bus_data는 data_B 출력
        en_A = 1; en_B = 1; #10; // 둘 다 활성화 -> 충돌 상황 (테스트에서 확인)
        en_A = 0; en_B = 0; #10; // 둘 다 비활성화 -> bus_data는 고임피던스 상태

        $finish;
    end
endmodule
```

LAB1 : Waveform

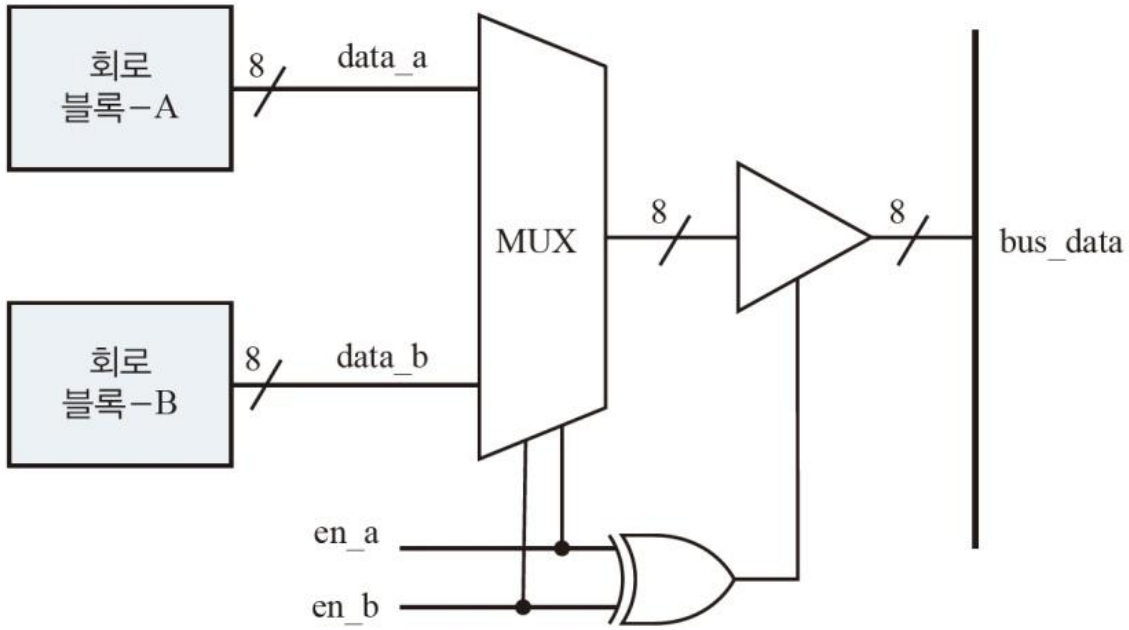
시뮬레이션 결과 예시

Time(ns)	en_A	data_A	en_B	data_B	bus_data
0	0	10101010	0	11001100	Z
10	1	10101010	0	11001100	10101010
20	0	10101010	1	11001100	11001100
30	1	10101010	1	11001100	X (충돌)
40	0	10101010	0	11001100	Z



LAB2 : Specification

- Multiplexed Bus Driver : Tri State Bus의 충돌방지를 구현



[그림 10-18] Multiplexed 버스 드라이버

LAB2 : DUT

```
1   `timescale 1ns / 1ps
2
3   module multiplexed_bus_driver (
4       input [7:0] data_a,     // 회로 블록 A의 데이터
5       input [7:0] data_b,     // 회로 블록 B의 데이터
6       input en_a,             // 회로 블록 A의 활성화 신호
7       input en_b,             // 회로 블록 B의 활성화 신호
8       output [7:0] bus_data   // 공통 데이터 버스
9   );
10
11      wire select;            // XOR 게이트 출력 (MUX 선택 신호)
12      wire [7:0] mux_out;     // MUX 출력
13
14      // XOR 게이트로 활성화 신호 결합
15      assign select = en_a ^ en_b;
16
17      // MUX 구현: 선택 신호에 따라 data_a 또는 data_b 선택
18      assign mux_out = (en_a) ? data_a :
19                       (en_b) ? data_b :
20                       8'bz;  // 고임피던스 상태
21
22      // 3상태 버퍼 구현: select 신호에 따라 MUX 출력 전달
23      assign bus_data = (select) ? mux_out : 8'bz;
24
25  endmodule
```

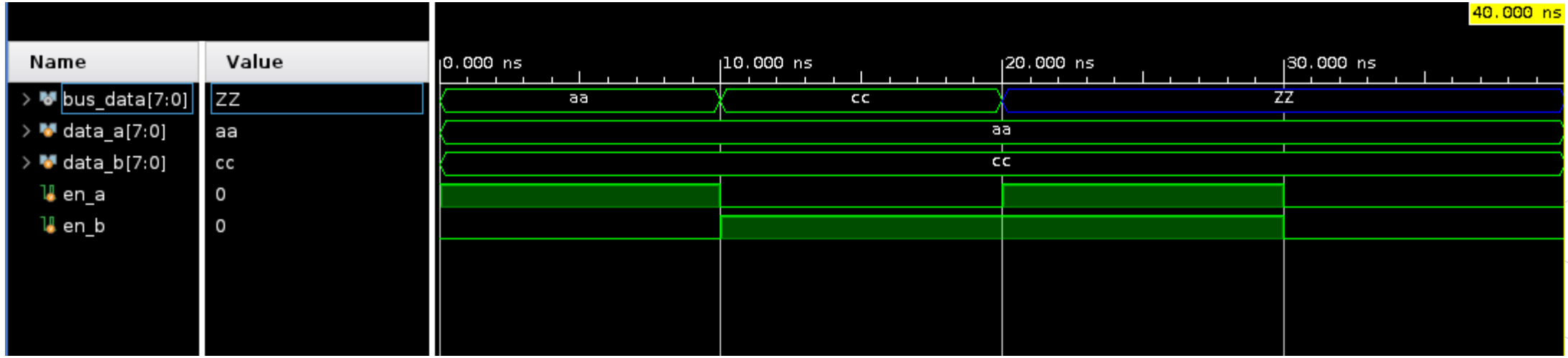
LAB2 : Testbench

```
`timescale 1ns / 1ps

module tb_multiplexed_bus_driver();
    reg [7:0] data_a;       // 테스트용 회로 블록 A 데이터 입력
    reg [7:0] data_b;       // 테스트용 회로 블록 B 데이터 입력
    reg en_a;               // 테스트용 회로 블록 A 활성화 신호
    reg en_b;               // 테스트용 회로 블록 B 활성화 신호
    wire [7:0] bus_data;    // 공통 데이터 버스 출력
    // DUT(Design Under Test) 인스턴스 생성
    multiplexed_bus_driver uut (
        .data_a(data_a),
        .data_b(data_b),
        .en_a(en_a),
        .en_b(en_b),
        .bus_data(bus_data)
    );
    initial begin
        $monitor("Time=%0t | en_a=%b, data_a=%b | en_b=%b, data_b=%b | bus_data=%b",
                 $time, en_a, data_a, en_b, data_b, bus_data);
        // 초기값 설정
        data_a = 8'b10101010;
        data_b = 8'b11001100;

        // 테스트 케이스들
        en_a = 1; en_b = 0; #10; // A 활성화 -> bus_data = data_a (10101010)
        en_a = 0; en_b = 1; #10; // B 활성화 -> bus_data = data_b (11001100)
        en_a = 1; en_b = 1; #10; // XOR 조건 위반 -> bus_data는 고임피던스 상태(Z)
        en_a = 0; en_b = 0; #10; // 둘 다 비활성화 -> bus_data는 고임피던스 상태(Z)
        $finish;
    end

endmodule
```

LAB2 : Waveform

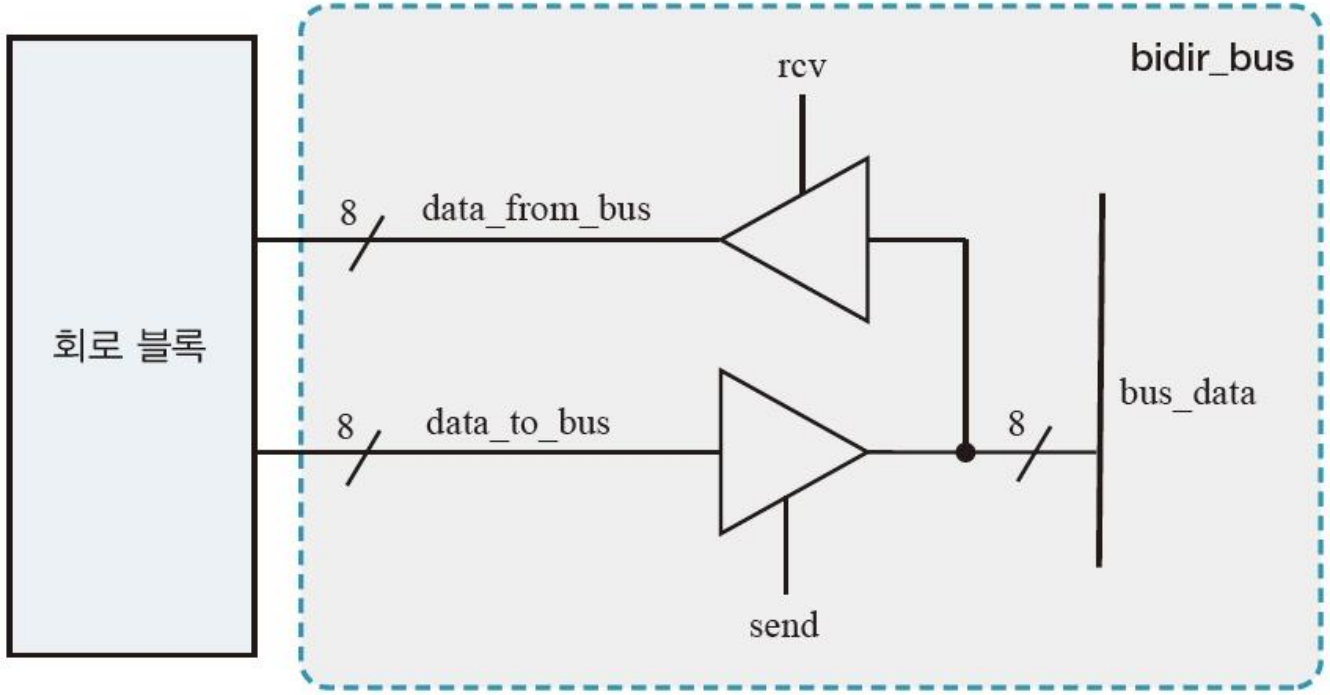
시뮬레이션 결과 예시

Time(ns)	en_a	data_a	en_b	data_b	bus_data
0	1	10101010	0	11001100	10101010
10	0	10101010	1	11001100	11001100
20	1	10101010	1	11001100	Z
30	0	10101010	0	11001100	Z



LAB3 : Specification

- Bidirectional Bus Driver
  - 보내기와 받기를 동시에 처리
  - Bus_data Port는 inout으로 선언



[그림 10-16] 8비트 양방향 버스 드라이버

LAB3 : DUT

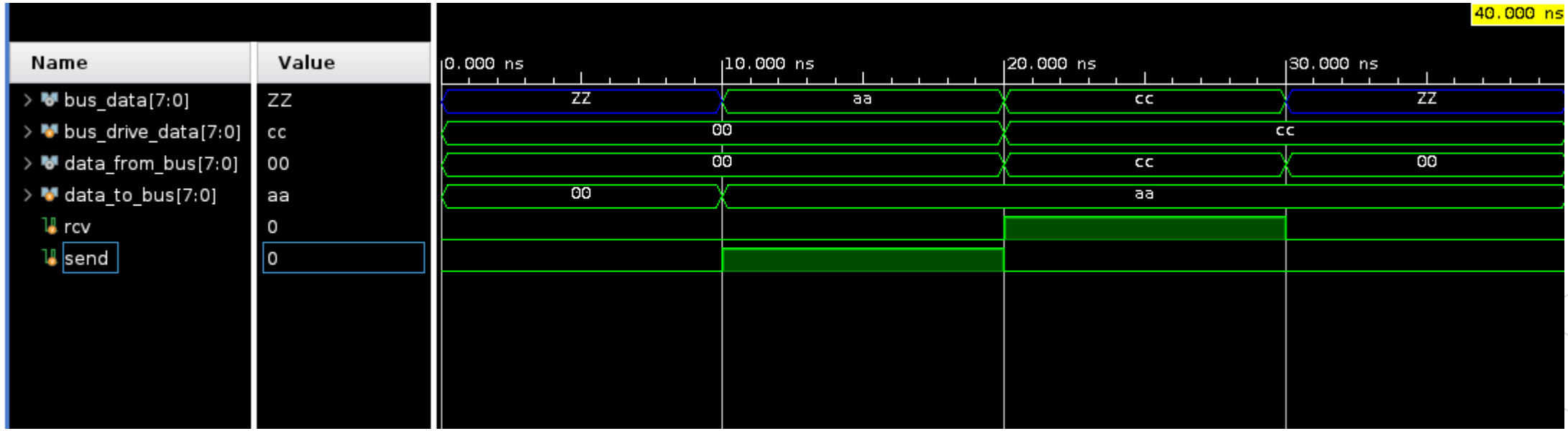
```
1   `timescale 1ns / 1ps
2
3   module bidirectional_bus_driver (
4       input [7:0] data_to_bus,      // 회로 블록에서 버스로 보낼 데이터
5       input send,                   // 데이터 전송 활성화 신호
6       input rcv,                    // 데이터 수신 활성화 신호
7       inout [7:0] bus_data,         // 양방향 데이터 버스
8       output reg [7:0] data_from_bus // 버스에서 읽어온 데이터
9   );
10
11      // 3상태 버퍼 구현: send 신호가 활성화되면 data_to_bus를 bus_data로 전달
12      assign bus_data = (send) ? data_to_bus : 8'bz;
13
14      // 데이터 수신 로직: rcv 신호가 활성화되면 bus_data를 data_from_bus로 읽어옴
15      always @(*) begin
16          if (rcv) begin
17              data_from_bus = bus_data;
18          end else begin
19              data_from_bus = 8'b0; // 기본값 설정
20          end
21      end
22
23  endmodule
```

LAB3 : Testbench

```
`timescale 1ns / 1ps

module tb_bidirectional_bus_driver();

    reg [7:0] data_to_bus;        // 송신 데이터 입력
    reg send;                     // 송신 활성화 신호
    reg rcv;                      // 수신 활성화 신호
    wire [7:0] bus_data;          // 양방향 데이터 버스 (inout)
    wire [7:0] data_from_bus;     // 수신 데이터 출력
    // DUT(Design Under Test) 인스턴스 생성
    bidirectional_bus_driver uut (
        .data_to_bus(data_to_bus),
        .send(send),
        .rcv(rcv),
        .bus_data(bus_data),
        .data_from_bus(data_from_bus)
    );
```

```
    // 강제로 bus_data를 구동하기 위한 레지스터
    reg [7:0] bus_drive_data;
    assign bus_data = (rcv) ? bus_drive_data : 8'bz;  // rcv가 활성화되면 외부에서 값을 구동
    initial begin
        $monitor("Time=%0t | send=%b | rcv=%b | data_to_bus=%b | bus_data=%b | data_from_bus=%b",
                 $time, send, rcv, data_to_bus, bus_data, data_from_bus);

        // 초기 상태 설정
        data_to_bus = 8'b00000000;
        send = 0;
        rcv = 0;
        bus_drive_data = 8'b00000000;
        #10;
        // 송신 테스트 (send 활성화)
        send = 1;
        rcv = 0;
        data_to_bus = 8'b10101010; #10;
        // 수신 테스트 (rcv 활성화)
        send = 0;
        rcv = 1;
        bus_drive_data = 8'b11001100; #10;
        // 둘 다 비활성화된 상태 확인 (고임피던스 상태)
        send = 0;
        rcv = 0; #10;
        $finish;
    end
endmodule
```

LAB3 : Waveform

시뮬레이션 결과 예시

Time(ns)	send	rcv	data_to_bus	bus_data	data_from_bus
0	0	0	00000000	Z	00000000
10	1	0	10101010	10101010	Z
20	0	1	XXXXXXXXXX	11001100	11001100
30	0	0	XXXXXXXXXX	Z	Z